

Web Application Authentication Assessment

Fares Zuleta

January 5, 2026

Scenario 1. *SecureMint Innovations has requested a security assessment of a newly deployed web application. The application introduces a new authentication mechanism and a strict password policy intended to improve account security.*

The objective of this assessment is to identify weaknesses in the authentication process that could allow unauthorized access. The analysis focuses on enumeration, brute-force techniques, and authentication bypasses using tools such as Burp Suite and ffuf.

1 Accessing the Web Application

When the target system is deployed, it provides an IP address. Entering this IP address into a web browser displays the main login page of the application.

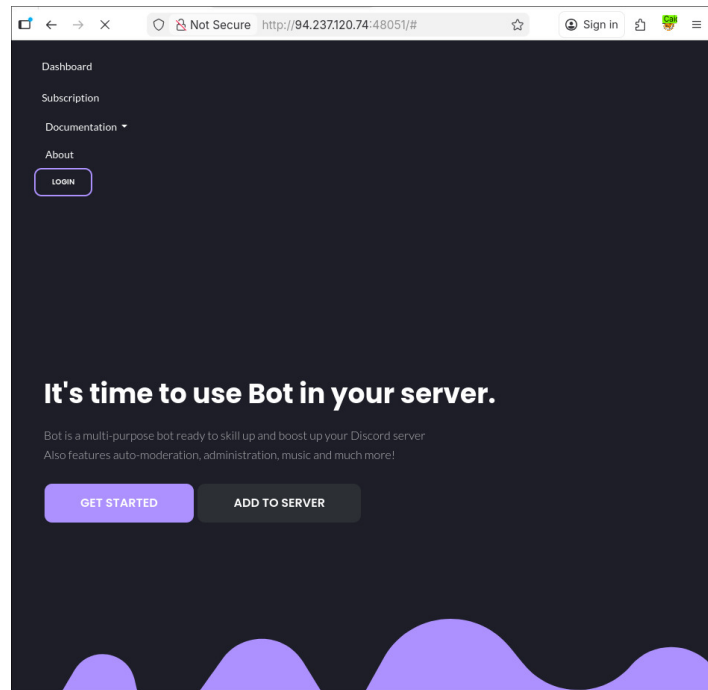


Figure 1: Login page of the web application

Attempting to log in redirects the user to a different directory. Above the login button, a link is provided to create a new account, indicating that user registration is enabled.

A screenshot of a web browser displaying a login page. The browser's address bar shows the URL `http://94.237.120.74:48051/login.php`. The page has a dark blue background. At the top left is a hamburger menu icon. The main heading is "Login to your account" in white. Below it are two input fields: "Username" with placeholder text "Enter username" and "Password" with placeholder text "Password". Below the password field is a link "Register a new account" in orange. At the bottom is a purple "SUBMIT" button.

Figure 2: User registration link

During account creation, the application enforces a password policy that requires the password to meet specific complexity requirements.

A screenshot of a web browser displaying a registration page. The browser's address bar shows the URL `http://94.237.120.74:48051/register.php`. The page has a dark blue background. At the top left is a hamburger menu icon. The main heading is "Register a new account" in white. Below it is a red error box with the text "Password does not meet our password policy:" followed by a bulleted list of requirements: "Contains at least one digit", "Contains at least one lower-case character", "Contains at least one upper-case character", "Contains NO special characters", and "Is exactly 12 characters long". Below the error box are two input fields: "Username" with placeholder text "Enter username" and "Password" with placeholder text "Password". Below the password field is a link "Back to login" in orange. At the bottom is a purple "SUBMIT" button.

Figure 3: Password policy enforcement

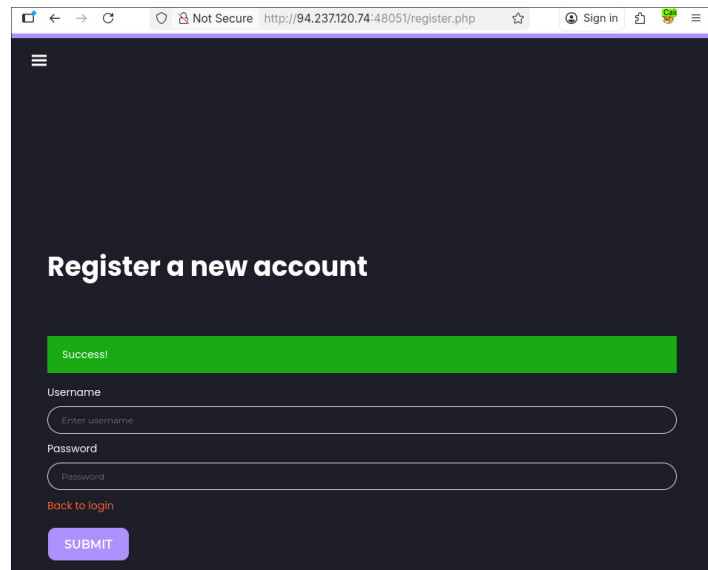


Figure 4: Account successfully created

Before logging in, the authentication error messages were tested. Submitting an invalid username returns the message “*Unknown username or password*”.

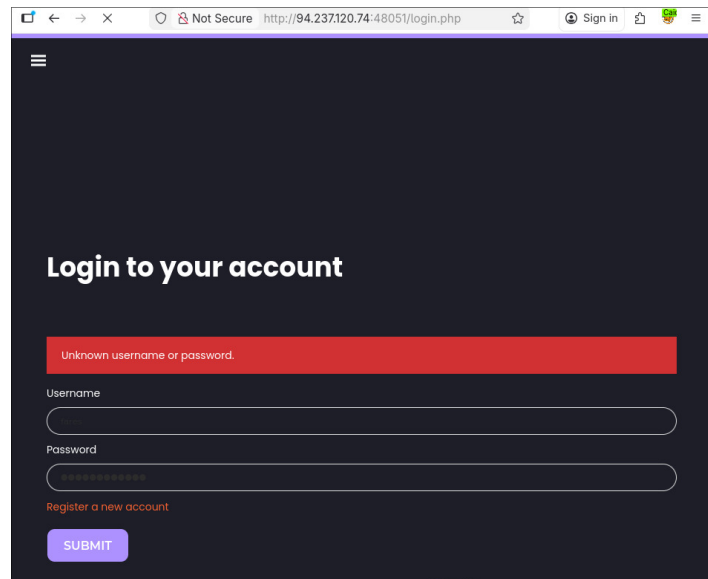


Figure 5: Error message for invalid username

However, when a valid username and an incorrect password are submitted, a different error message is displayed.

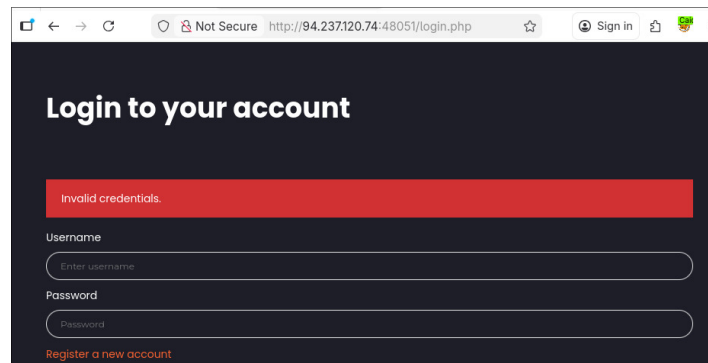


Figure 6: Error message for invalid password

This behavior confirms a username enumeration vulnerability. After submitting valid credentials, access to the user account is granted.

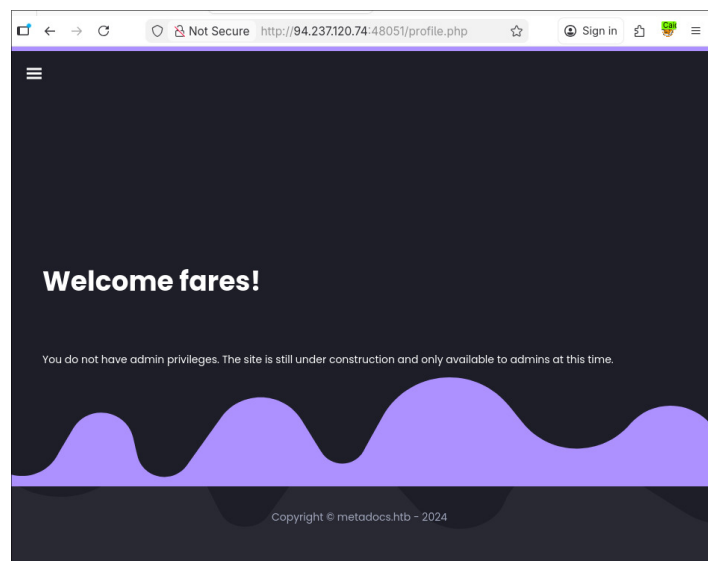


Figure 7: Successful login

2 Step 2: Username and Password Brute Force

To exploit the enumeration vulnerability, both ffuf and Burp Suite were used. ffuf was preferred for brute-force attacks due to its speed.

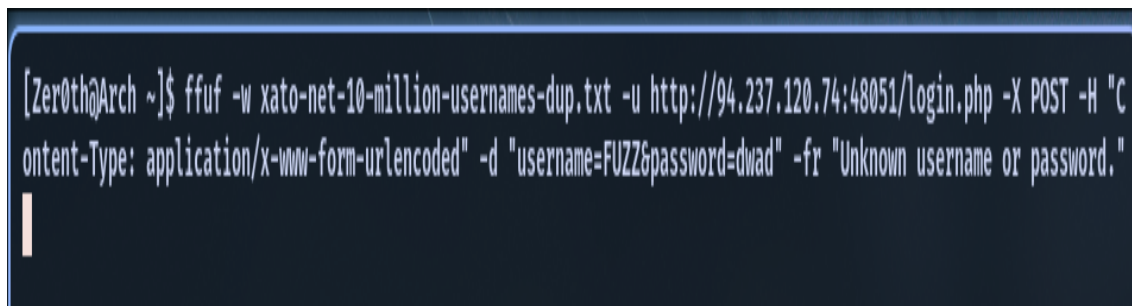


Figure 8: Username brute-force attack using ffuf

A publicly available list containing 10 million common usernames was used. Once a valid username was identified, the brute-force process was stopped.

```
v2.1.0-dev

:: Method      : POST
:: URL         : http://94.237.120.74:48051/login.php
:: Wordlist    : FUZZ: /home/Zer0th/Desktop/Lists/SecLists/Usernames/xato-net-10-million-usernames-dup.txt
:: Header     : Content-Type: application/x-www-form-urlencoded
:: Data       : username=FUZZ&password=dwad
:: Follow redirects : false
:: Calibration : false
:: Timeout     : 10
:: Threads    : 40
:: Matcher    : Response status: 200-299,301,302,307,401,403,405,500
:: Filter     : Regexp: Unknown username or password.

gladys [Status: 200, Size: 4344, Words: 680, Lines: 91, Duration: 166ms]
:: Progress: [9859/624370] :: Job [1/1] :: 244 req/sec :: Duration: [0:00:44] :: Errors: 0 ::
```

Figure 9: Valid username discovered

Using the same approach, a brute-force attack was launched against the password parameter by fuzzing the password field.

```
[Zer0th@rocky:~]$ ffuf -u http://94.237.120.74:48051/login.php -H POST -H 'Content-Type: application/x-www-form-urlencoded' -d 'username=gladys&password=FUZZ' -fr 'Invalid credentials.'

v2.1.0-dev

:: Method      : POST
:: URL         : http://94.237.120.74:48051/login.php
:: Wordlist    : FUZZ: /home/Zer0th/Desktop/Lists/SecLists/Passwords/rockyou.txt
:: Header     : Content-Type: application/x-www-form-urlencoded
:: Data       : username=gladys&password=FUZZ
:: Follow redirects : false
:: Calibration : false
:: Timeout     : 10
:: Threads    : 40
:: Matcher    : Response status: 200-299,301,302,307,401,403,405,500
:: Filter     : Regexp: Invalid credentials.

#0x0f00000 [Status: 200, Size: 0, Words: 1, Lines: 1, Duration: 163ms]
:: Progress: [13378/12962] :: Job [1/1] :: 243 req/sec :: Duration: [0:00:52] :: Errors: 0 ::
```

Figure 10: Password brute-force attack

3 Step 3: Two-Factor Authentication Bypass

After successful authentication, the application redirects the user to a two-factor authentication (2FA) page. In many cases, the OTP could be brute-forced; however, a directory bypass was identified.

Upon successful login, the application normally redirects authenticated users to `/profile.php`, indicating that this resource exists.

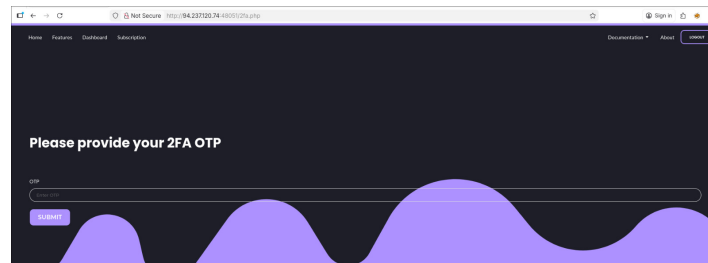


Figure 11: 2FA verification page

Intercepting the request using Burp Suite reveals the next server interaction.

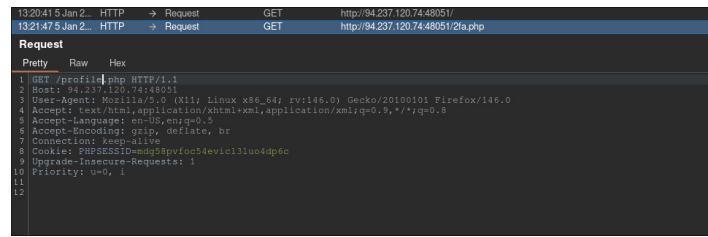


Figure 12: Intercepted request in Burp Suite

The request was sent to Burp Intruder, and the directory was manually modified to `/profile.php`. The server initially returned a permission error.

By manipulating the response header to return 200 OK, access to the protected resource was granted.

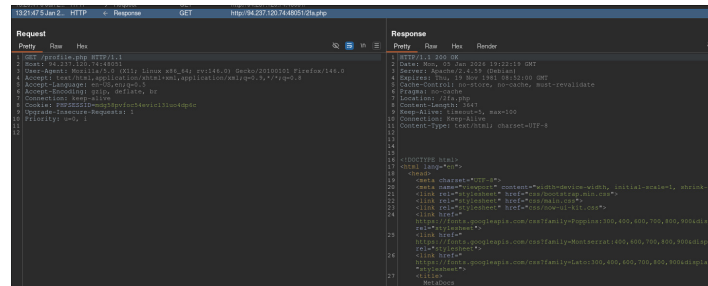


Figure 13: Modified response header

As a result, unauthorized access to the profile page was achieved, successfully bypassing the 2FA mechanism and revealing the flag.

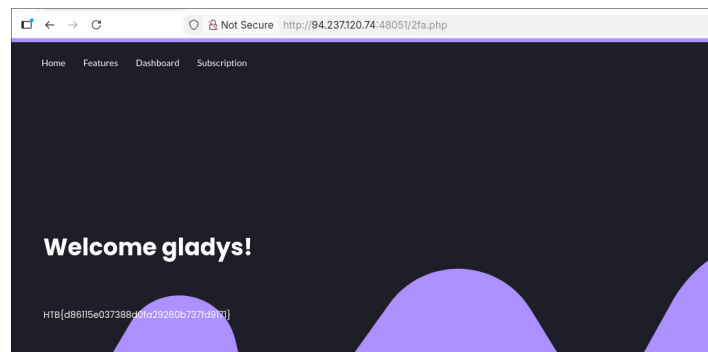


Figure 14: Flag obtained